

الدليل الشامل لإدارة نافذة السياق وترشيد التوكنز في Claude Code

الدليل العملي لحوكمة الجلسات وخفض خط الأساس لمشروع Blind App

يقدم هذا المستند استراتيجية هندسية متكاملة لإدارة نافذة السياق (Context Window) في Claude Code، وتصحيح المفاهيم الشائعة حول أمر ضغط الجلسة (/compact)، مع خطوات خفض خط الأساس (Baseline) وقواعد النظافة اليومية لتوفير كوتا الحساب وتسريع وتيرة الإنتاج البرمجي.

أولاً: تشريح المشكلة — لماذا يعتبر "compact عند 40%" فكرة خاسرة تماماً؟

يعتقد كثير من المطورين أن تنفيذ أمر /compact بمجرد وصول مؤشر استهلاك الجلسة إلى 40% يحميهم من تجاوز سقف 200 ألف توكن. هذا التصور غير صحيح عملياً وهندسياً للأسباب التالية:

طبيعة خط الأساس الثابت (The Baseline):

عند فتح أي جلسة جديدة أو بعد تنفيذ أمر التصفير الشامل (/clear)، لا يبدأ مؤشر الجلسة من الصفر أبداً، بل يبدأ فعلياً من قرابة 39% (ما يعادل 78,000 توكن محجوزة مسبقاً لتعليمات النظام، وملف CLAUDE.md، وأدوات الـ MCP، والمهارات المحقونة).

ضغط جلسة فارغة من العمل الحقيقي:

تنفيذ أمر /compact عند وصول المؤشر إلى 40% يعني أنك تضغط جلسة لم تستهلك سوى 1% فقط (نحو 2,000 توكن) من العمل الفعلي! لا يوجد محتوى لاختصاره، بل تفقد التفاصيل اللحظية دون تحقيق أي وفر حقيقي.

أمر compact لا ينزل تحت خط الأساس إطلاقاً:

آلية عمل /compact تقوم على حذف سجل المحادثة، وصياغة ملخص، ثم إعادة تحميل خط الأساس كاملاً (78k) مضافاً إليه حجم الملخص. أي أن الحد الأدنى بعد أي compact هو 40% فأكثر.

التكلفة العكسية واستهلاك التوكنز المضاعف (Double Penalty):

كل عملية compact تكلف توكنز: النموذج يدفع توكنز كتابة لتوليد الملخص، ثم يضطر لإعادة قراءة الملفات التي سقط سياقها، مما يزيد الاستهلاك بدلاً من خفضه.

ثانياً: الحقيقة الأهم — أين تكمن المشكلة الحقيقية في استهلاك الكوتا؟

سقف الـ 200k ليس مشكلتك؛ لأن Claude Code يطبق ضغطاً ذاتياً (Auto-Compact) عند 95%. الخطر الفعلي هو نفاذ كوتا الحساب الأسبوعية وسرعة بلوغ حدود الاستهلاك (Rate Limits). سبب استنزاف الكوتا يرجع إلى بنية استدعاء النماذج اللغوية:

في كل تفاعل أو أمر طرفي أو Tool Call، يعيد المحرك إرسال كامل نافذة السياق السابقة.

الحسبة الرياضية الصادمة: إذا كانت جلستك منتفخة بحجم 150 ألف توكن، وقام الوكيل بتنفيذ 20 أداة طرفية (Tool Calls) متتالية لفحص الكود وبناءه، فإنك تستهلك: 150k مضروبة في 20 دورة = 3 ملايين توكن تم حرقها في مهمة واحدة فقط!

الخلاصة المعمارية: الاستراتيجية الصحيحة ليست مراقبة رقم مئوي مجرد، بل تقليل الحجم الإجمالي لما يُعاد إرساله في كل دورة (Turn).

ثالثاً: الركيزة الأولى — خفض خط الأساس (أكبر مكسب فوري دائم)

الخطوة الأكثر تأثيراً هي تجريد خط الأساس (78k) من المكونات الزائدة التي لا يستخدمها مشروع Blind App. هناك قرابة 40,000 توكن مهدرة تُحَقَّن قسرياً في كل دورة:

المكون المهدر	الحجم التقريبي	الإجراء والتوصية الهندسية
موصلات claude.ai و MCP Connectors (أكثر من 250 أداة: Figma, Notion, Supabase, Canva, Drive)	12k - 15k توكن	تعطيل الموصلات السحابية غير المستخدمة من واجهة claude.ai أو عبر قائمة mcp/
إضافة ECC (تحقق 90 وكيلاً و 330 مهارة في قائمة الأدوات)	~12k توكن	تعطيلها في settings.local.json لمشروع Blind App وتفعيلها عند الطلب فقط
إضافات المتصفح وفحص الويب (chrome-devtools plugin)	~3k - 5k توكن	تعطيلها في مشاريع تطبيقات الهواتف التي لا تتطلب اختبار واجهات ويب
قارئ المستندات (pdf-mcp)	~1k توكن	الإبقاء عليه فقط في حال قراءة ملفات ومستندات PDF داخل الجلسة

الأثر المالي والتقني المباشر:

ينخفض خط الأساس فوراً من 39% (78k~) إلى 23% - 25% (~45k)، موفراً أكثر من 30,000 توكن في كل Turn طوال اليوم، وهو مكسب دائم يتكرر في كل تفاعل ويحمي كوتا الحساب من النفاد المبكر.

رابعاً: الركيزة الثانية — عتبة التصفير الاستراتيجي (/strategic-clear) عند 70% إلى 75%

دورة العمل المعيارية (Milestone Cycle): في مشروع Blind App، دورة العمل المتزنة هي إنجاز أمر أو أمرين (كتابة ميزة، تشغيل الفحوصات، ثم Commit). هذه الدورة تستهلك بين 40k إلى 60k توكن.

لماذا /strategic-clear/ أفضل هندسياً من /compact/؟

الحالة الوحيدة المسموح فيها باستخدام /compact/: في منتصف بوابة فحص طويلة (Quality Gate) ولا تريد خسارة حالة الأدوات اللحظية، بشرط إرفاقه بتعليمات مقيدة صارمة: compact keep the order id, the gate step reached, and the exact failing/test names.

خامساً: الركيزة الثالثة — قواعد النظافة اليومية داخل الجلسة

حظر طباعة مخرجات Gradle الخام: مخرجات Gradle تلتهم بين 5k إلى 15k توكن في المرة الواحدة. القاعدة: توجيه المخرجات لملف وفحصها عبر الفترة: gate.log > ... gradlew ثم قراءة السطور المطلوبة فقط.

القراءة المقتضبة للملفات: الاعتماد على sed -n أو head أو grep -n بدل قراءة الملفات كاملة. وفي CURRENT_STATE.md يُقرأ القسم الأول فقط.

تقارير الوكلاء المنفصلة: يكتب OpenCode تقريره في fleet_orders/reports، ويُقرأ منه قسم الفروقات (Variance) والخلاصة التنفيذية فقط.

حظر تفريع الوكلاء (Subagents): كل وكيل فرعي يبدأ من الصفر ويعيد تحميل وحقق خط الأساس بالكامل.

تجميد ملفات الإعدادات أثناء الجلسة: تعديل CLAUDE.md أو settings أثناء الجلسة النشطة يبطل الكاش (38k) ويعيد حساب التفاعلات بالسعر الكامل. عدّلها قبل clear/ دائماً.

سادساً: الركيزة الرابعة — أسرار التوفير: وقت الفراغ والرسائل المجمعة

وقت السكون مجاني 100%: السياق في وضع الانتظار لا يستهلك شيئاً. الجلسة عند 60% لا تكلف فلساً واحداً أثناء توقفك، والاستهلاك يحدث فقط عند إرسال رسالة.

قاعدة الرسالة الشاملة: تجميع 5 استفسارات في رسالة واحدة منسقة يوفر عليك 4 دورات كاملة من حجم السياق السابق، مما يضاعف العمر التشغيلي للكوتا.

سابعاً: الخلاصة التنفيذية وقائمة الإجراءات العملية

تعديل إعدادات المشروع: تعطيل إضافة ECC في `claude/settings.local.json`.
.`ecc@ecc: false`

تعطيل الموصلات السحابية: كتابة `mcp/` في Claude Code وتعطيل الموصلات غير المستخدمة، أو تعطيلها من واجهة `.claude.ai`.

ضبط مؤشر السياق: التحقق من ضبط `env:CLAUDE_STATUSLINE_CTX_WINDOW` ليعكس الحجم الفعلي بدقة.

اعتماد التصفير عند **70-75%**: بعد كل Commit ناجح، تجميد الحالة وتنفيذ التصفير النظيف لبدء أمر جديد بكامل التركيز.

ثامناً: أمر التشغيل والضبط المباشر لـ Claude Code

لتطبيق هذه الاستراتيجية آلياً داخل Claude Code، يتم إرسال التوجيه المعياري التالي:
قم بتعديل ملف `settings.local.json` داخل مجلد `.claude`. في هذا المشروع، وعطّل إضافة ECC
بجعل خاصية `ecc@ecc` بقيمة `false`. ثم تأكد من توجيه مخرجات Gradle إلى `gate.log` بدلاً من
طباعتها في الكونسول، واعتمد التصفير عند بلوغ 70% إلى 75% مع تجميد الإنجازات في
`.CURRENT_STATE.md`.

.